

Date	15 March 2026
Team ID	XXXXXX
Project Title	VBCUA — Voice-Based Concept Understanding Analyser
Maximum Marks	5 Marks

PROJECT PROPOSAL (PROPOSED SOLUTION)

PROJECT OVERVIEW

Objective	The primary objective is to build an automated application designed to evaluate spoken technical conceptual explanations, providing instant quantitative scoring and qualitative feedback on alignment and delivery pace.
Scope	Encompasses audio file handling, automated transcription pipeline integration, semantic text alignment metrics, real-time audio statistics tracking, local relational history databases, and summary report exports.

PROBLEM STATEMENT

Description	Traditional computer science screening assessments focus exclusively on written text, code syntax, or multiple-choice verification. This methodology fails to measure how confidently and correctly a trainee or candidate can vocally explain technical architecture or algorithmic topics, leaving vocal filler habits and structural understanding gaps completely undetected.
Impact	Solving this lets learners find gaps in their technical speech, control speech anxiety, eliminate repetitive filler phrases (um, uh), and build strong speaking confidence needed for multi-stage technical technical interviews.

PROPOSED SOLUTION

Approach	Using an integrated local machine learning script sequence. It transcribes explanations through Whisper STT, compares the generated text structurally against verified topic references using Sentence-BERT word vector embeddings, extracts pause lengths and pacing attributes via Librosa, and synthesizes analytical feedback templates through the Google Gemini model.
Key Features	• Automated Multi-Engine Scoring: Dynamic composite weighing logic splitting evaluations into 60% semantic accuracy and 40% delivery speed/fluency fluency scoring. • Interactive Diagnostics Panel: Streamlit-powered local GUI featuring immediate visual waveforms, speech pace metrics (WPM tracking), and exact vocal filler word tallies.

- **Automated Reporting and Archiving:** Instant storage mapping using clean local SQLite relational tables alongside fully formatted PDF report sheets.

RESOURCE REQUIREMENTS

Resource Type	Description	Specification/Allocation
Hardware	Local Development Workspace Machine Machine CPU / Standard Execution Architecture	Modern Multi-Core Processor (x86_64 or ARM64)
Memory	System memory allocated for processing libraries and machine learning model checkpoints	Minimum 8 GB RAM (16 GB Recommended)
Storage	Local disk requirements for audio caching, SQLite DB, and local report document generations	256 GB SSD Storage Workspace
Software (Framework)	Main user-facing layout web dashboard interface library layer	Streamlit Framework (Python)
Software (Libraries)	Machine learning tokenizers, vector math utilities, acoustics handlers, database mappers	openai-whisper, sentence-transformers, librosa, soundfile, google-generativeai, SQLAlchemy, pandas
Development Environment	Integrated Development Environment setting for module tracking	VS Code / PyCharm / Virtualenv
Data Checkpoints	Pre-trained word embedding models and speech-to-text weight architectures running locally	all-MiniLM-L6-v2 (SBERT), Whisper-base model weights